

Pull Requests

👋 Intros

Software is a team sport. Without one another we will be left to our own blind spots and missteps. Together, though, we can accomplish some unbelievably great things.

One of the best ways to do this is to have a good, high-signal PR process. We know that code doesn't just get yeeted up to the cloud (or Flexential 😊).

It requires an engineer to understand a problem, hunt for an appropriate solution that satisfies not only that particular problem but an incredibly high amount of *other* problems (security, availability, traceability, etc etc-ility), and then implement it while dancing through hundreds of files, thousands of considerations, and tens of thousands lines of code.

TLDR: we need each other. Like the African proverb teaches:

Go fast? Go alone. Go far? Go together.

💡 PRs are one of the daily critical-path means by which we go far together, so it serves us well to be intentional with how we do so.

💡 Core Principles

- 1. **Be respectful and constructive** - Focus on the problem & solution, not the person
- 2. **Be specific and actionable** - Vague feedback has never once been helpful
- 3. **Prioritize issues** - Distinguish between blockers and nice-to-haves
- 4. **Share knowledge** - Use reviews as teaching opportunities
- 5. **Be timely** - Respond to PRs within one business day

🔍 Review Process IN REVIEW

Every PR gets **1 required reviewer**. This will be assigned via Round Robin in Github. An SME can be added as optional or in place of R1. Finally, we always post to the group for crowd participation.

Note: **participation** doesn't have to mean *contribution*, it is just as often for the sake of **education** for those reading the PR.

Roles	How	Purpose	Expectations	SLAs
✍️ Author	Create a PR	Getting eyes on the proposed solution in order to provide feedback critical to a high bar for quality & completeness	• A PR that follows the agreed upon minimum completeness and requirements (e.g. tests,	Responsiveness to the work a Reviewer puts in to the PR, usually within the same 1-2 business day TAT that

			updated README, code standards, etc) • Receptivity to comments and questions raised by Reviewers	Reviewers are working to meet
🔴 R1 - Round Robin	Auto-assigned from HS & ROIP Review team	Load distribution, first-pass review	• Can approve if feeling good about their own understanding • Can defer to R2 if needing further expertise	Some response: 1 business day. Can negotiate beyond that. Ex: “Overall looks good, but I'll need another day to get everything setup to test locally.” Escalation: if ghosted or stale, bump in Slack and/or ask for someone to sub in
🧠 R2: SME	Called in directly by Author or R1 if determined to be needed	Deeper knowledge of inner workings, domain, context, business constraints, technology, etc. Ex: K8s related PR causes Author to add Don; Michael reviews an AJS related PR but decides to call in BenK for an extra measure of thoroughness	• Get quicker review from someone recently involved in {some domain} or resident SME. • Provide deeper review than the R1 believes they can give.	1-2 business days
👥 RN: the Teams	Author posts to #roi-zone-all or a project channel	Allows folks to opt-in for learning and context building. A fail mode to be avoided: too many cooks in the kitchen & dogpiling of reviews.	Follow along, only add if high value and missed by other reviewers.	N/A

✖ Exceptions:

- **OOO/PTO:** Auto-skip in rotation, author reassigns R2
- **Large PRs (>500 LOC):** Discuss timeline with reviewer first
- **Hotfixes:** Flag in Slack for immediate attention (< 2hrs)

💬 PR Comment Types

Use these prefixes to categorize your comments:

Prefix	Moj	Description	Action Required
[Blocker]	✖🧑	Critical issue that must be fixed before merging	Yes
[Required]	⚠️	Important change needed before approval	Yes
[Suggestion]	💡	Recommended improvement but not required	No
[Question]	?	Request for clarification	Response needed
[Nit]	🔧	Minor stylistic or cosmetic issue	No
[FYI]	🗣️	Educational comment with no action needed	No

📋 Review Structure

👤 For PR Submitters

1. **PR Title:** Concise, descriptive, includes ticket number (e.g., "HS-123: Fix correspondence table index performance")
2. **PR Description Template:**

```
1 ## Overview
2 [Brief description of the changes]
3
4 ## Problem
5 [What issue does this solve?]
6
7 ## Solution
8 [How does this PR solve the problem?]
9
10 ## Testing
11 [How was this tested?]
12
13 ## Screenshots/Logs
14 [If applicable]
15
16 ## Related PRs
17 [If applicable]
18
```

3. Self-review checklist:

- [] Code follows team coding standards [TODO]
- [] Tests added/updated
- [] Documentation updated as needed

🧑 For Reviewers

1. **Overall Assessment:** Start with a summary comment that covers:
 - General impression of the PR
 - Major concerns (if any)
 - Areas of praise
2. **Line-by-line feedback:** Use the comment prefixes above
3. **Approval Statuses:**
 - **✅ Approve:** No issues found or only minor suggestions
 - **💬 Comment:** Questions or suggestions that don't block approval
 - **🧑 Request Changes:** Blockers or required changes identified

- [] No unnecessary commented code
- [] No debug print statements

📌 Comment Quality Guidelines

Don't be a turd. Especially on accident!

High quality outcomes require high quality code which requires high quality *feedback*.

Be generous with both the quality and the quantity.

👉 Effective Comments

✅ Explain why, not just what

```
1 [Suggestion] Consider using a prepared
  statement here instead of string
  concatenation to prevent SQL injection
  vulnerabilities.
2
```

✅ Provide examples

```
1 [Required] This query might cause performance
  issues with large datasets. Consider adding
  an index on the `correspondence_date` column:
2
3 ALTER TABLE correspondence
4 ADD INDEX idx_corr_date
  (correspondence_date);
5
```

✅ Link to resources

```
1 [FYI] We've already got a great method for
  handling this: [link to docs]
2
```

✅ Ask clarifying questions

```
1 [Question] Was there a specific reason for
  choosing JSONB over a structured column here?
  Not sure I'm understanding, help me out
  please!
2
```

😬 Code-Specific Reminders

🗄 Database Changes

- Indexes for query performance
- Transaction handling
- Migration rollback plans
- Security considerations (permissions, injection risks)
- Data validation

🖥 Backend Changes

- Error handling
- Input validation
- Security considerations
- Performance implications
- Logging

🤖 Ineffective Comments

❌ Vague criticism

```
1 This doesn't look right.
2
```

❌ Personal preferences without context

```
1 I prefer a different approach here.
2
```

❌ Nitpicking without marking as such

```
1 Move this variable declaration up 3 lines.
2
```

❌ Assumptive comments

```
1 You didn't test this, did you?
2
```

❌ REALLY vague criticism

```
1 No.
```

👤 Frontend Changes

- Accessibility considerations
- Responsive design
- Error state handling
- Loading state handling
- Browser compatibility

👤 Miscellany

- Clear code beats clever code
- Documentation
- Tests
- Good commenting
- Consistency in design pattern
- Linting & formatting

Examples of Excellent Reviews

Example 1: Database Schema Review

```
1  **Overall Assessment:**
2  Good work on the correspondence table updates. The schema changes look well-structured and follow our naming conventions. I have a few
3  suggestions around indexing and one required change regarding nullability.
4  [Required] The 'correspondence_date' column should be NOT NULL since we always require a date for correspondence records. This will help
5  maintain data integrity and improve query performance.
6  [Suggestion] Consider adding an index on (user_id, correspondence_date) as this seems to be a common query pattern in the application
7  based on our metrics.
8  [Question] Was there a specific reason for making the 'metadata' column JSONB instead of creating structured columns? If we know the
9  structure in advance, it might be better for querying.
10 [FYI] For future reference, our team has created a schema design checklist here: [link]
11
12 otherwise...LGTTTTMM 
13
```

Example 2: Code Performance Review

```
1  **Overall Assessment:**
2  This PR gets the job done for the ticket, but I've got some hesitations about it being performant.
3
4  [Blocker] The query in `fetchAllCorrespondence()` is not paginated. It could cause memory issues, especially w larger data sets like
5  we're dealing with.
6  [Required] The N+1 query issue in the correspondence loop needs a second pass. Consider using a bulk fetch approach instead:
7
8  ```sql
9  SELECT * FROM correspondence_attachments
10 WHERE correspondence_id IN (SELECT id FROM correspondence WHERE user_id = xx)
11 ```
12
13 [Suggestion] Consider memoizing the permission check since it's in the loop
14
15 [Nit] I'm not sold on the FooBar class name, it's admittedly close but feels like it could fall prey to misdirection.
16
17 Happy to approve once the blocker and required changes are addressed!
```

 Appendix

 Tooling considerations

 [GitHub Code Reviews With Pull Request Channels | Axolo](#)

 Git Workflow Standards

 [ROI Git Workflow](#)